

Uniwersytet Wrocławski

Generatory Liczb Pseudolosowych

Autorzy: Łukasz Stodółka, Łukasz Besuch, Marcin Rypula

Wrocław, 2026

Contents

1	What is a Pseudorandom Number Generator?	2
1.1	Key Properties of PRNGs	2
1.2	Seed (Initial Value)	2
1.3	State Space	3
1.4	Period	3
1.5	Uniformity (Distribution)	3
1.6	Summary	3
2	Classical Methods of Pseudorandom Number Generation	4
2.1	Linear Congruential Generator (LCG)	4
2.1.1	Period of LCG	4
2.1.2	Properties and Limitations	5
2.1.3	Example of LCG	5
2.2	Lehmer Generator	5
2.2.1	Properties	5
2.2.2	Advantages and Disadvantages	6
2.3	Combined Generators	6
2.3.1	Period	6
2.3.2	Advantages	6
2.4	Summary	6
3	What is a Recursive Generator?	7
3.1	Linear Feedback Shift Register (LFSR)	8
3.2	Maximum Length Sequence:	8
3.3	The Mathematics of LFSR	8

1 What is a Pseudorandom Number Generator?

A pseudorandom number generator (PRNG) is an algorithm that produces a sequence of numbers which approximates the properties of random numbers.

Unlike true randomness, which is based on unpredictable physical phenomena, a PRNG is entirely deterministic. This means that if the initial conditions are known, the entire sequence can be reproduced exactly.

In a more formal way, a PRNG can be defined as a sequence (x_n) generated by a recurrence relation:

$$x_{n+1} = f(x_n)$$

where f is a deterministic function and x_0 is the seed.

PRNGs are widely used in simulations, cryptography, numerical analysis, and computer science applications where random-like behavior is required.

1.1 Key Properties of PRNGs

In order for a pseudorandom number generator to be useful, it should satisfy several important properties:

- **Determinism:** The generator always produces the same sequence from the same initial value.
- **Efficiency:** The algorithm should be fast and require minimal computational resources.
- **Statistical Randomness:** The generated sequence should pass statistical tests for randomness, meaning it should not exhibit obvious patterns.

1.2 Seed (Initial Value)

The seed is the starting value used by a PRNG. It determines the entire sequence of generated numbers.

Formally, the seed x_0 is the initial element of the state space S , and the generated sequence satisfies:

$$x_n \in S, \quad n \geq 0$$

If the same seed is used multiple times, the generator will produce exactly the same sequence. This property is useful for reproducibility in simulations and debugging.

Different seeds lead to different sequences, increasing variability.

1.3 State Space

A PRNG operates on a finite state space S . Since S is finite and the generator is deterministic, every sequence must eventually become periodic.

1.4 Period

The period of a PRNG is the length of the sequence before it starts repeating.

Formally, the smallest positive integer T such that:

$$x_{n+T} = x_n \quad \text{for all } n$$

is called the period of the generator.

A good generator has a very long period, making repetition practically unnoticeable in real applications.

1.5 Uniformity (Distribution)

Uniformity refers to how evenly the generated numbers are distributed over a given interval.

For a good PRNG, numbers should be uniformly distributed, meaning that each value (or range of values) appears with approximately the same frequency over time.

Formally, for a sequence (x_n) in the interval $[0, 1]$, uniformity means that for any subinterval $[a, b] \subset [0, 1]$:

$$\lim_{N \rightarrow \infty} \frac{1}{N} \# \{x_1, x_2, \dots, x_N \in [a, b]\} = b - a$$

Lack of uniformity can lead to biased results, which is particularly problematic in simulations and statistical applications.

1.6 Summary

A good pseudorandom number generator should produce sequences that are deterministic yet appear random, have a long period, and exhibit uniform distribution. These properties ensure that the generator can be reliably used in practical applications.

Many practical PRNGs, such as linear congruential generators or LFSR, are based on specific choices of the function f and the structure of the state space.

2 Classical Methods of Pseudorandom Number Generation

Classical pseudorandom number generators are based on simple mathematical formulas, typically involving modular arithmetic and recurrence relations. Despite their simplicity, they form the foundation for many modern generators.

In this section, we present three important types of classical generators: the Linear Congruential Generator (LCG), the Lehmer generator, and combined generators.

2.1 Linear Congruential Generator (LCG)

The Linear Congruential Generator is one of the oldest and simplest PRNGs. It generates a sequence of numbers using the recurrence relation:

$$x_{n+1} = (ax_n + c) \mod m$$

where:

- x_n is the current state,
- a is the multiplier,
- c is the increment,
- m is the modulus,
- x_0 is the seed.

The sequence (x_n) takes values in the set $\{0, 1, 2, \dots, m - 1\}$.

2.1.1 Period of LCG

The maximum possible period of an LCG is m . This is achieved if and only if the following conditions are satisfied:

- $\gcd(c, m) = 1$,
- $a - 1$ is divisible by all prime factors of m ,
- if m is divisible by 4, then $a - 1$ is also divisible by 4.

These conditions are known as the Hull-Dobell Theorem.

2.1.2 Properties and Limitations

LCGs are easy to implement and computationally efficient. However, they suffer from several drawbacks:

- relatively short periods for poorly chosen parameters,
- visible patterns in higher dimensions,
- limited statistical quality compared to modern generators.

2.1.3 Example of LCG

Example with parameters: $m = 9$, $a = 2$, $c = 1$, $x_0 = 1$

n	x_n
0	1
1	3
2	7
3	6
4	4
5	0
6	1

Table 1: Example sequence generated by LCG

2.2 Lehmer Generator

The Lehmer generator is a special case of the LCG where the increment $c = 0$. Its recurrence relation is:

$$x_{n+1} = (ax_n) \mod m$$

It is also known as the multiplicative congruential generator.

2.2.1 Properties

The Lehmer generator has a simpler structure than the general LCG, but requires careful choice of parameters.

To achieve a long period, the modulus m is often chosen as a prime number, and the multiplier a should be a primitive root modulo m .

In that case, the generator achieves the maximum period:

$$T = m - 1$$

2.2.2 Advantages and Disadvantages

- simpler structure than LCG,
- good statistical properties for well-chosen parameters,
- still vulnerable to patterns and predictability.

2.3 Combined Generators

Combined generators are constructed by combining two or more simpler generators (such as LCGs) in order to improve statistical properties and increase the period.

A typical combined generator may take the form:

$$x_n = (x_n^{(1)} - x_n^{(2)}) \mod m$$

where $x_n^{(1)}$ and $x_n^{(2)}$ are sequences generated by two independent generators.

2.3.1 Period

If the individual generators have periods T_1 and T_2 , the combined generator can achieve a period close to:

$$T \approx \text{lcm}(T_1, T_2)$$

where lcm denotes the least common multiple.

2.3.2 Advantages

- significantly longer period,
- improved statistical properties,
- reduced correlation between generated values.

2.4 Summary

Classical generators such as LCG and Lehmer are simple and efficient but have limitations in terms of statistical quality. Combined generators address some of these issues by increasing the period and improving randomness properties.

These methods form the basis for understanding more advanced pseudorandom number generators.

3 What is a Recursive Generator?

A recursive generator is an algorithm that creates sequence of numbers where the next number is calculated using the previous number (or several previous numbers).

Every recursive generator relies on:

- **The Seed (Initial State):** The generator needs a starting point to begin the process. If you give a recursive generator the exact same starting seed, it will always produce the exact same sequence of numbers.
- **The State:** This is the current “memory” of the generator. As the generator produces new numbers, its state updates.

Because the generator relies on its previous state to create the next one, the process loops continuously. The goal of a good recursive generator is to make this loop so long and complicated that the sequence of numbers appears completely unpredictable to an outside observer.

3.1 Linear Feedback Shift Register (LFSR)

It is incredibly fast, operates fundamentally on binary numbers (0s and 1s), and is widely used in digital electronics and simple cryptography. LFSR consists of:

1. **Shift Register:** A device that shifts contents into adjacent positions in register or out of register if position is on the end. The positions where the content was is left empty if there is no content that that could be shifted there.
2. **Output Bit:** The bit at the end that is moved out of a shift register.
3. **Feedback Function:** A function that takes bits from certain positions "**taps**", combines them in some sort of function and the result is fed back into the register's input bit. Bits are collected before the shift.

3.2 Maximum Length Sequence:

An n-bit LFSR has 2^n possible states. However, because a state of all zeros will infinitely produce zeros, the maximum possible period for an LFSR sequence is $2^n - 1$.

3.3 The Mathematics of LFSR

In mathematical terms, the new bit b_n is calculated recursively based on the previous bits. If we have a register of length k , and the taps are defined by coefficients c_i (where c_i is 1 if there is a tap, and 0 if there isn't), the formula looks like this:

$$b_n = (c_1b_{n-1} + c_2b_{n-2} + \dots + c_kb_{n-k}) \pmod{2}$$

(Note: In binary math, addition modulo 2 is exactly the same thing as the XOR operation).

This relationship is represented by a characteristic polynomial:

$$P(x) = 1 + c_1x + c_2x^2 + \dots + c_nx^n$$

To achieve the maximum period of $2^n - 1$, the characteristic polynomial $P(x)$ must be a primitive polynomial over $GF(2)$.

Example of LFSR for 4-bit number with tap sequence [3, 4] and **XOR** being the feedback function:

a	b	c (Tap)	d (Tap)	Output
1	0	0	1	1
1	1	0	0	0
0	1	1	0	0
1	0	1	1	1
0	1	0	1	1
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1
0	1	1	1	1
0	0	1	1	1
0	0	0	1	1
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
1	0	0	1	1

1